UX Engineering y Accesibilidad Web aplicada a Web-Apps A11y

Aplicar una metodología básica de **diseño centrado en usuario** al proyecto Web-App.
Definir y evaluar **flujos de usuario** (tasks) y detectar fricción con heurísticas y métricas cualitativas.
Implementar accesibilidad base y media (teclado, foco, ARIA mínima, formularios, alt, contraste) de forma justificable.
Utilizar herramientas de auditoría (WAVE/Axe/Lighthouse) para generar evidencias en la documentación técnica.
Diseño centrado en usuario · WCAG/ARIA

UX en ingeniería: definición operativa

UX = resultado de:

- **utilidad** (sirve para algo real)
- **usabilidad** (es fácil y eficiente)
- **accesibilidad** (lo puede usar más gente)
- **confianza** (claridad, feedback, errores comprensibles)

Idea clave: UX no es “opinión”: se diseña y se evalúa.

Restricciones reales en Web-Apps

En proyectos reales, UX se diseña con restricciones:

- tiempo y coste
- compatibilidad de navegadores
- latencia/red
- rendimiento en móvil
- diversidad de usuarios

Modelo base: Usuario · Tarea · Contexto

Para definir UX correctamente:

- **Usuario:** perfil, motivación, capacidades (incl. diversidad funcional)
- **Tarea:** qué quiere hacer, con qué frecuencia, con qué urgencia
- **Contexto:** móvil en calle, escritorio en casa, mala conexión, etc.

Resultado: decisiones razonadas, no estéticas.

A11y

El término **a11y** es un numerónimo que significa **accesibilidad** (del inglés *accessibility*). Se abrevia de esta manera porque existen **11 letras** entre la "a" inicial y la "y" final de la palabra en inglés.

En el ámbito del desarrollo web profesional, la accesibilidad (a11y) no es solo una característica adicional, sino un **requisito central** de calidad y cumplimiento legal para garantizar experiencias digitales inclusivas.

Implementar a11y significa diseñar y construir interfaces que puedan ser utilizadas por todas las personas, incluyendo aquellas con discapacidades visuales, motoras o cognitivas. Esto implica:

- **Cumplimiento de estándares:** Seguir las pautas **WCAG 2.2** (Web Content Accessibility Guidelines), que incluyen reglas sobre el contraste de color, el tamaño de los elementos interactivos y la visibilidad del foco del teclado.
- **Uso de HTML Semántico:** Utilizar etiquetas HTML de forma correcta (como <main>, <nav>, <button>) y recurrir a atributos **ARIA** solo cuando los elementos nativos no sean suficientes para comunicar la función a las tecnologías asistivas (como lectores de pantalla).
- **Respeto a las preferencias del usuario:** Utilizar consultas de medios (media queries) para detectar si el usuario prefiere **movimiento reducido** (prefers-reduced-motion) o temas de color específicos (como el modo oscuro), adaptando la interfaz en consecuencia.
- **Diseño para la navegación:** Asegurar que todos los elementos sean operables mediante teclado, que el orden de tabulación sea lógico y que los objetivos táctiles tengan un tamaño mínimo (al menos **24x24 píxeles**) para evitar errores de activación.

a11y representa el esfuerzo de ingeniería para eliminar las barreras digitales y asegurar que la web sea un espacio accesible para todos los usuarios.

Persona vs “perfil técnico”

En el curso basta con 2–3 perfiles tipo:
- Usuario explorador (búsqueda general)
- Usuario decidido (entra con objetivo concreto)
- Usuario con necesidades de accesibilidad (teclado, lector, baja visión)

Entregable para P1: describir perfiles en 6–8 líneas.

Tareas (tasks) como unidad de diseño

Ejemplos de tareas típicas en la Web-App del curso:
- Buscar un ítem por texto
- Filtrar por: categoría, fecha, ubicación, …
- Ver detalle
- Guardar favorito
- Compartir / abrir mapa

Ingeniería: diseñamos la UI alrededor de tareas, no de “páginas”.

Flujos de usuario (User flows)

Un flow se representa como:
- Estado inicial
- Pasos (acciones)
- Estados intermedios (loading/empty/error)
- Estado final (success)

Ejemplo de flow (texto → filtro → detalle)

1. Home → abrir catálogo
2. introducir texto “festival”
3. aplicar filtro “música”
4. abrir card
5. detalle → “ver en mapa”

Estados obligatorios:

- loading (API)
- empty state (“no se encontraron resultados”)
- error state (“no se pudo cargar, reintentar”)

¿Qué es un “estado” en una Web-App?

Un **estado de interfaz** representa una situación concreta del sistema en un momento determinado.
Formalmente:
Estado = configuración observable de la interfaz + condición lógica del sistema.
Ejemplo simple:

- El usuario pulsa “Buscar”
- Se ejecuta una petición fetch()
- La interfaz cambia visualmente

No es lo mismo:

- Esperando resultados
- Mostrando resultados
- Mostrando error
- No hay resultados

Cada uno es un **estado distinto**.

Estados de interfaz (modelo formal)

Estados típicos en Web-Apps:

- Idle
- Loading
- Success
- Empty
- Error
- Offline (opcional)

Errores comunes en alumnos:

- no mostrar loading → el usuario cree que “no va”
- empty sin explicación → frustración
- error sin acción → callejón sin salida

1. Idle

Estado inicial o en reposo. No hay acción en curso.
Ejemplo práctico:

- Formulario vacío
- Página cargada sin interacción
- Pantalla inicial antes de buscar eventos culturales

2. Loading

El sistema está procesando o esperando datos (API, JSON, servidor).
Ejemplo típico:

setState("loading")

Visualmente: Texto “Cargando…”

En la práctica de la Web-App cultural esto ocurre cuando:

- Se filtran eventos
- Se consulta por isla
- Se accede a geolocalización
- Se consumen APIs externas

Error común de alumnos:
No modelar este estado → la interfaz “parece congelada”.

Success

La operación ha finalizado correctamente y hay datos válidos.
Ejemplo:

- Se muestran eventos filtrados
- Aparece un mapa con marcadores
- Se confirma reserva

Visualmente:

- Lista visible
- Mensaje de confirmación
- Botón activo nuevamente

Este estado debe estar claramente diferenciado del estado *Empty*.

Empty

La operación ha sido correcta, pero no hay resultados.
Diferencia clave:

Success	Empty
Hay datos	No hay datos
Lista visible	Mensaje explicativo

Ejemplo:
“No hay eventos culturales en Menorca en febrero.”
Este estado es clave en UX y se evalúa dentro de *Usabilidad*
Criteris avaluacio Web-App: Muchos proyectos fallan aquí mostrando simplemente una lista vacía.

Error

La operación falla (servidor, red, JSON mal formado, error JS).
Ejemplos:

- Error 404
- JSON inválido

- API externa caída
- Error de validación de formulario

Visualmente:

- Mensaje claro
- Color diferenciado
- Botón “Reintentar”

Debe cumplir:

- Mensaje explicativo
- No técnico
- No mostrar stack trace

Offline (opcional pero avanzado)

Especialmente relevante si el proyecto es:

- Responsive
- PWA
- Con Service Worker

Definición: El dispositivo no tiene conexión a Internet.
Visual:

- Banner “Sin conexión”
- Uso de datos cacheados
- Botón deshabilitado

Feedback: el principio más infrautilizado

El sistema debe indicar:

- qué está pasando
- qué puedo hacer ahora
- si mi acción tuvo efecto

Ejemplos:

- “Guardado en favoritos”
- “Cargando resultados…”
- “Sin conexión. Mostrando datos guardados.”

Microcopy de ingeniería (mensajes con precisión)

Mensajes eficaces:

- describen el problema
- explican causa probable (sin culpar)
- proponen acción concreta

Mal: “Error 400”
Bien: “No se pudo cargar la lista. Revisa tu conexión y reintenta.”
Evaluación: se valoran mensajes de error explicativos

Arquitectura de información

Preguntas clave:

- ¿qué entidades existen? (evento, lugar, artista, obra)
- ¿cómo se relacionan?
- ¿cómo navega el usuario? (lista→detalle, mapa→detalle, etc.)

Consecuencia técnica: modelado de JSON y rutas/vistas.

Ejemplo de estados

```
const STATES = {IDLE: "idle",LOADING: "loading",SUCCESS: "success",
 EMPTY: "empty", ERROR: "error", OFFLINE: "offline" };
```

```
let currentState = STATES.IDLE;
function setState(newState) {
  currentState = newState;
  render();
}
```

Y luego:

```
async function fetchData() {
try {
    setState(STATES.LOADING);
    const response = await fetch("/data.json");
    if (!response.ok) throw new Error();
    const data = await response.json();
    if (data.length === 0) {
      setState(STATES.EMPTY);
    } else {
      setState(STATES.SUCCESS);
    }
  } catch {
    setState(STATES.ERROR);
  }
}
```

Esto demuestra:

- Estructura lógica
- Modularidad
- Separación de responsabilidades

Criterio directamente evaluable según documento de evaluación

Diseño de navegación (patrones)

Patrones adecuados para este curso:

- top nav + secciones
- tabs
- drawer menu (móvil)
- breadcrumbs (opcional)
- buscador persistente en catálogo

Regla: navegación coherente y “predecible”.

Accesibilidad: definición operativa

Accesibilidad = que el sistema sea utilizable por:

- teclado
- lector de pantalla
- usuarios con baja visión / daltonismo
- usuarios con movilidad limitada
- usuarios en contexto adverso (móvil, sol, ruido)

Ingeniería: accesibilidad también mejora calidad general.

WCAG: idea base

Cuatro principios (POUR):

1. Perceptible
2. Operable
3. Comprensible
4. Robusto

Curso: aplicamos estos principios en decisiones concretas (foco, labels, contraste, semántica).

Checklist mínimo (lo exigible en el curso)

- HTML semántico (landmarks, headings correctos)
- Navegación completa por teclado
- Foco visible (`:focus-visible`)
- Imágenes con `alt` correcto
- Formularios con `label` asociado
- Contraste suficiente
- No depender solo del color
- `lang` correcto en documento

Todo esto aparece en criterios de evaluación

Teclado: reglas concretas

- Tab: recorre elementos interactivos en orden lógico
- Enter/Espacio: activan controles (botón)
- Escape: cierra modales (si los hay)
- Traps: evitar que el foco quede "atrapado"

Antipatrón: `div` clickable sin `tabindex` ni role.

Foco: por qué es crítico

- El foco es "dónde estoy" para teclado y lectores
- No se debe eliminar el outline sin alternativa
- El foco debe ser visible y con contraste

Ejemplo:

```
:focus-visible { outline: 2px solid currentColor; outline-offset: 2px; }
```

ARIA

Regla de oro:
- Usa HTML nativo primero
- ARIA para nombrar landmarks o controles sin texto

Casos típicos válidos:
- nav aria-label="Navegación principal"
- button aria-label="Abrir filtros" (si solo hay icono)

Evaluación: etiquetas ARIA se revisan

Accesibilidad y color: casos reales

- Errores no solo en rojo: añade icono + texto
- Estados activos con etiqueta visible ("Filtro activo")
- Enlaces distinguibles sin depender solo del color

Evaluación: contraste y colores

Motion y preferencias del usuario

Si hay animaciones/transiciones:
- respetar prefers-reduced-motion
- evitar parallax agresivo
- evitar animaciones largas que bloqueen interacción

Impacto: accesibilidad + rendimiento.

Herramientas de auditoría: qué son y para qué

- **WAVE:** reporta estructura, landmarks, alt, contrastes, headings
- **Axe (DevTools):** reglas automáticas WCAG, muy útil en desarrollo
- **Lighthouse:** auditoría general (performance + ux + best practices + SEO)

Limitación importante: herramientas ≠ verdad absoluta
- Detectan fallos automáticos, pero no evalúan:
  - claridad del flujo
  - calidad del microcopy
  - coherencia de navegación
- UX requiere test cualitativo (aunque sea simple)

Ingeniería: combinar métricas automáticas + revisión humana estructurada.

Evidencias: cómo documentar UX/A11y

En documentación técnica:
- capturas de auditoría (antes/después)
- lista de issues encontrados
- correcciones aplicadas
- justificación de decisiones (trade-offs)

Esto refuerza nota en documentación técnica

Caso real (aplicado al proyecto)

Escenario: usuario filtra eventos, API tarda 2s

- sin loading: usuario repite acciones → duplicación de requests
- con loading + disabled button: control y claridad

Conclusión: UX bien diseñada reduce bugs y carga.

Errores típicos de proyectos (y cómo evitarlos)

- demasiado texto en home (sin jerarquía)
- filtros confusos
- no existe "volver a resultados"
- navegación distinta entre vistas
- layout móvil con controles demasiado pequeños
- falta de estados "empty/error"

Requisitos P1

Incluir

- definición de usuarios (2–3 perfiles)
- definición de tareas principales
- 2 flows clave con estados (loading/empty/error)
- estructura de navegación (mapa de pantallas)
- decisiones de accesibilidad base (lista de checklist)

Alineación con evaluación P1 (diseño gráfico + UI + funcionalidades planteadas)

Checklist evaluable (orientado a criterios del curso)

- Interfaz intuitiva
- Flujo de trabajo lógico
- Mensajes de error explicativos
- Formularios y call-to-action eficaces
- Accesibilidad: ARIA, teclado, alt, contraste
- Responsive usable (no solo "se adapta")